

# A Machine-Checked Zero-Knowledge Proof-of-Provenance Pipeline for Skill Circuits

with a description-logic reading of the development itself

Formalized in Lean 4 with Mathlib

Project modules `RequestProject.ZkPoP.*` and `RequestProject.ZkPoPDL`

## Abstract

We report a complete, machine-checked formalization of a zero-knowledge proof-of-provenance (zkPoP) pipeline for *skill circuits*: small programs whose well-typedness is to be certified to a third party without revealing the program or its witness. The development is written in Lean 4 on top of Mathlib, contains no `sorry`, and the axiom closure of every headline theorem consists of Lean’s three standard axioms only.

The pipeline has five phases, each with its own correctness theorem: a typed object language with a checker and an evaluator, joined by a semantic bridge theorem; an arithmetization of that checker into a multivariate polynomial system, proved to be satisfiable exactly for the terms the checker accepts; a Tseitin-style flattening of polynomial systems into gate circuits, proved to preserve satisfiability; a verified gadget catalog; Pedersen commitments with perfect hiding — unconditional in a prime-order group — and binding under an explicitly formalized discrete-logarithm assumption, together with a Schnorr  $\Sigma$ -protocol with completeness and special soundness by an explicit extractor; a self-hosting universal verifier closed under recursion and library aggregation; and Reed–Solomon/Shamir sharding with proofs of durability and privacy. The two headline results are an end-to-end theorem — a term accepted by the checker yields a satisfied arithmetization, a satisfied flat artifact, and acceptance under two levels of recursive universal verification — and its converse, that a satisfied artifact certifies a well-typed skill.

We then apply to the development the same treatment that an earlier part of this project applied to a poem: the pipeline itself is rendered as a knowledge base in the description logic  $\mathcal{ALC}$ , shown consistent by an explicit finite model, with machine-checked entailments (hiding, no-junk, two-level recursion, axiom-closure audit, dual use) and machine-checked non-entailments (possibility is not deed). A final section states plainly what is *not* proved.

## Contents

|          |                                                    |          |
|----------|----------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                                | <b>2</b> |
| 1.1      | The problem . . . . .                              | 2        |
| 1.2      | Contributions . . . . .                            | 2        |
| 1.3      | The rooster in the room . . . . .                  | 3        |
| 1.4      | Conventions . . . . .                              | 3        |
| <b>2</b> | <b>The object language and the semantic bridge</b> | <b>4</b> |
| 2.1      | Types, tags, wires . . . . .                       | 4        |
| 2.2      | Terms, checker, evaluator . . . . .                | 4        |
| <b>3</b> | <b>Phase I: arithmetizing the checker</b>          | <b>4</b> |
| 3.1      | The relational form . . . . .                      | 4        |
| 3.2      | The polynomial form . . . . .                      | 5        |

|           |                                                                |           |
|-----------|----------------------------------------------------------------|-----------|
| <b>4</b>  | <b>Phase II: polynomial systems, flattening, gate circuits</b> | <b>5</b>  |
| <b>5</b>  | <b>The verified gadget catalog</b>                             | <b>6</b>  |
| <b>6</b>  | <b>Phase III: commitment and extraction</b>                    | <b>6</b>  |
| 6.1       | Hiding, unconditionally . . . . .                              | 6         |
| 6.2       | Binding as a security statement . . . . .                      | 7         |
| <b>7</b>  | <b>Phase IV: self-hosting and recursion</b>                    | <b>8</b>  |
| <b>8</b>  | <b>Phase V: deployment, durability, audit</b>                  | <b>8</b>  |
| 8.1       | Sharding . . . . .                                             | 8         |
| 8.2       | The end-to-end theorem . . . . .                               | 9         |
| 8.3       | The audits . . . . .                                           | 9         |
| 8.4       | A worked skill . . . . .                                       | 9         |
| 8.5       | Axiom closure . . . . .                                        | 9         |
| <b>9</b>  | <b>The development as a description-logic knowledge base</b>   | <b>10</b> |
| 9.1       | Why . . . . .                                                  | 10        |
| 9.2       | The TBox . . . . .                                             | 10        |
| 9.3       | Consistency . . . . .                                          | 11        |
| 9.4       | Entailments . . . . .                                          | 12        |
| 9.5       | Non-entailments . . . . .                                      | 12        |
| 9.6       | Reality check . . . . .                                        | 12        |
| <b>10</b> | <b>What is <i>not</i> proved</b>                               | <b>13</b> |
| <b>11</b> | <b>Conclusion</b>                                              | <b>13</b> |
| <b>A</b>  | <b>Module map</b>                                              | <b>14</b> |

# 1 Introduction

## 1.1 The problem

A *skill* is a small program, and a *skill circuit* is an artifact published about that program. The engineering question this development answers is: what exactly does a party learn when it accepts such an artifact, and what exactly is guaranteed to be true when it does?

The usual answer is a diagram: checker  $\rightarrow$  arithmetization  $\rightarrow$  circuit  $\rightarrow$  commitment  $\rightarrow$  proof  $\rightarrow$  verifier. Each arrow of that diagram is a claim, and claims of this kind are exactly where deployed systems fail: an arithmetization that accepts strictly more than the checker; a flattening that introduces a scratch wire an adversary can steer; a “universal verifier” whose satisfiability is not actually equivalent to the inner claim; a commitment whose hiding argument was never stated precisely enough to be checked. This paper replaces the diagram by theorems.

## 1.2 Contributions

- 1. A semantic bridge (§2).** An object language of types and terms, a checker `Tm.infer`, an evaluator `Tm.eval`, and the theorem `Tm.checker_sound`: in a well-formed environment, whatever the checker accepts at type  $A$  evaluates to a value of type  $A$ . Integration is gated on this theorem, and it is proved.

2. **Arithmetization with an exact correspondence** (§3). A relational arithmetization `typeSat` and a genuine polynomial system `tagSys`, each proved satisfiable *exactly* for the terms the checker accepts (`typeSat_iff`, `tagSys_sat_iff`), with a canonical satisfying assignment for completeness and determinism of the tag wire.
3. **Flattening that preserves satisfiability** (§4). A Tseitin-style compilation of polynomials into gates with an explicit linear scratch-wire discipline, and `flatSys_sat_iff`.
4. **A verified gadget catalog** (§5): zero-testing with an inverse hint, multiplexing, Horner evaluation, root membership, and Merkle path checking, each with a satisfiability characterization.
5. **Commitments and extraction** (§6): Pedersen perfect hiding — unconditional in a prime-order group, in both the unique-opening and the distributional form — binding as an explicit discrete-logarithm reduction *and* as a security statement under an explicitly formalized hardness assumption, and Schnorr special soundness with an extractor that returns the witness.
6. **Self-hosting** (§7): a universal circuit whose satisfiability is equivalent to the inner claim, recursion at any depth, library aggregation, and demonstrations that the universal verifier accepts a valid inner assignment and rejects an invalid one.
7. **Deployment and audit** (§8): Reed–Solomon durability, Shamir privacy, the end-to-end theorem and its converse, the combined privacy and binding audits, and an axiom-closure table.
8. **The development as a knowledge base** (§9): the pipeline rendered in the description logic  $\mathcal{ALC}$ , proved consistent by a finite model, with entailments and non-entailments.
9. **An honest limitations section** (§10).

### 1.3 The rooster in the room

This development sits inside a larger project that formalized a cycle of poems — *Ode to Coq: The Meta-Rooster* and its sequels — as description-logic knowledge bases. The cycle’s subject is a proof assistant conscripted: a bird whose crow once welcomed the sun and now verifies missile firmware, and whose lament is that “each theorem proved, each type secure, ensures destruction ever more.”

Section 9 takes that seriously rather than decoratively. The same  $\mathcal{ALC}$  machinery used on the poem is turned on this pipeline, and one of the poem’s own axioms — a mathematical object *can serve the sword* or *can serve the brief green silence* — is asserted of our own artifact and its consequence proved:

$$\text{Artifact} \sqsubseteq \exists \text{canServe.Sword} \sqcap \exists \text{canServe.GreenSilence}.$$

The counterpart non-entailment is proved too: the knowledge base does *not* entail that the artifact serves the sword. Possibility is not deed. A verifier settles what is true, not what it is for; the second question is left where it belongs, with the people who deploy it.

### 1.4 Conventions

All statements below are Lean declarations. The circuit field is an arbitrary field  $\mathbb{F}$ ; where tags of compound types must be distinguishable we assume characteristic zero, and concrete artifacts are instantiated at  $\mathbb{Q}$ . Group operations are written additively in an  $\mathbb{F}$ -module  $G$ , as elliptic-curve groups are used in practice. Wire assignments are functions  $\mathbb{N} \rightarrow \mathbb{F}$ .

## 2 The object language and the semantic bridge

### 2.1 Types, tags, wires

**Definition 2.1** ( $\text{ZkPoP.Ty}$ ). Object-language types are generated by

$$A, B ::= \text{nat} \mid \text{bool} \mid A \rightarrow B \mid \text{vec } n \ A.$$

A circuit cannot speak about inductive types; it speaks about field elements. Two encodings are used.

**Definition 2.2** ( $\text{Tag}$ ).  $\text{tag} : \text{Ty} \rightarrow \mathbb{N}$  is defined by  $\text{tag}(\text{nat}) = 0$ ,  $\text{tag}(\text{bool}) = 1$ ,  $\text{tag}(A \rightarrow B) = 2 + 2 \langle \text{tag } A, \text{tag } B \rangle$  and  $\text{tag}(\text{vec } n \ A) = 3 + 2 \langle n, \text{tag } A \rangle$ , where  $\langle \cdot, \cdot \rangle$  is Cantor pairing — so arrows get even tags above 1 and vectors odd ones. Its field version is  $\text{tag}_{\mathbb{F}}(A) = (\text{tag } A : \mathbb{F})$ .

**Theorem 2.3** ( $\text{Ty.tag\_injective}$ ,  $\text{Ty.tagF\_injective}$ ). *tag is injective, and  $\text{tag}_{\mathbb{F}}$  is injective whenever  $\mathbb{F}$  has characteristic zero.*

**Definition 2.4** (Flat wire encoding).  $\text{wires} : \text{Ty} \rightarrow \text{List } \mathbb{N}$  is the prefix serialization:  $\text{wires}(\text{nat}) = [0]$ ,  $\text{wires}(\text{bool}) = [1]$ ,  $\text{wires}(A \rightarrow B) = 2 :: \text{wires}(A) ++ \text{wires}(B)$  and  $\text{wires}(\text{vec } n \ A) = 3 :: n :: \text{wires}(A)$ .

**Theorem 2.5** ( $\text{Ty.wires\_injective\_flat}$ ). *wires is injective. The proof goes through the stronger prefix lemma  $\text{Ty.wires\_append\_inj}$ : if  $\text{wires}(A) ++ xs = \text{wires}(B) ++ ys$  then  $A = B$  and  $xs = ys$ ; that is, the serialization is uniquely decodable.*

**Remark 2.6** (Index-normal form). Because indices are evaluated,  $\text{vec } (2 + 3) \ A$  and  $\text{vec } 5 \ A$  receive literally the same encoding:  $\text{Ty.wires\_index\_normal}$  and  $\text{Ty.tag\_index\_normal}$  are proved by `rfl`.

### 2.2 Terms, checker, evaluator

**Definition 2.7** ( $\text{ZkPoP.Tm}$ ). Terms are variables, numeric literals, bits, `add`, `mul`, `isZero` and `ite`. Values are `Val.nat` and `Val.bool`, with `ty` returning `nat` or `bool`.

The checker  $\text{infer} : \text{List Ty} \rightarrow \text{Tm} \rightarrow \text{Option Ty}$  and the evaluator  $\text{eval} : \text{List Val} \rightarrow \text{Tm} \rightarrow \text{Option Val}$  are the obvious recursive functions. An environment is well-formed,  $\text{EnvOk } \rho \ \Gamma$ , when  $\text{map ty } \rho = \Gamma$ .

**Theorem 2.8** (Semantic bridge;  $\text{Tm.checker\_sound}$ ). *If  $\text{EnvOk } \rho \ \Gamma$  and  $\text{infer } \Gamma \ t = \text{some } A$ , then there is a value  $v$  with  $\text{eval } \rho \ t = \text{some } v$  and  $\text{ty}(v) = A$ .*

This is the non-negotiable statement of the whole pipeline: everything the circuits certify is a statement about `infer`, and Theorem 2.8 is what makes a statement about `infer` a statement about running the program.

## 3 Phase I: arithmetizing the checker

### 3.1 The relational form

**Definition 3.1** ( $\text{ZkPoP.typeSat}$ ).  $\text{typeSat}_{\mathbb{F}} \ \Gamma \ t \ \tau$  is defined by recursion on  $t$ : it assigns one *tag wire* value  $\tau$  per subterm and imposes one constraint per constructor. For instance a variable requires  $\Gamma[i] = A$  and  $\tau = \text{tag}_{\mathbb{F}}(A)$ ; a literal requires  $\tau = \text{tag}_{\mathbb{F}}(\text{nat})$ .

**Theorem 3.2** (Gate shape).  *$\text{typeSat\_add\_gate}$ :  $\text{typeSat}(\text{add } a \ b) \ \tau$  holds iff there are  $t_a, t_b$  with  $\text{typeSat } a \ t_a$ ,  $\text{typeSat } b \ t_b$  and  $t_a = t_b = \tau = 0$  — the arithmetic constructors force their inputs to carry the `nat` tag.  $\text{typeSat\_ite\_gate}$ :  $\text{typeSat}(\text{ite } c \ t \ e) \ \tau$  holds iff the condition tag is 1, the branch tags are equal, and  $\tau$  is their common value.*

**Theorem 3.3** (Exact correspondence; `typeSat_iff`). *For all  $t$  and  $\tau$ ,*

$$\text{typeSat}_{\mathbb{F}} \Gamma t \tau \iff \exists A, \text{infer } \Gamma t = \text{some } A \wedge \tau = \text{tag}_{\mathbb{F}}(A).$$

**Corollary 3.4** (`typeSat_unique`). *The tag wire is determined: if  $\text{typeSat } t \tau$  and  $\text{typeSat } t \sigma$  then  $\tau = \sigma$ .*

### 3.2 The polynomial form

The relational form is not yet a circuit. `ZkPoP.tagEqs` produces an actual list of multivariate polynomials over  $\mathbb{F}$ , and `tagSys $_{\mathbb{F}}$   $\Gamma t n$`  packages them as a `PolySystem`. Tag wires are laid out in preorder, so the subtree of  $t$  based at  $n$  occupies exactly the window  $[n, n + |t|)$ :

**Lemma 3.5** (`tagEqs_varsIn`, `tagSys_bounded`). *Every polynomial of  $\text{tagSys } \Gamma t n$  has all its variables in  $[n, n + |t|)$ ; in particular the system is bounded by  $n + |t|$ .*

Soundness is `tagSys_sound`; completeness is witnessed by an explicit canonical assignment `canon` built by `Function.update` along the same preorder traversal (`canon_sat`). Together:

**Theorem 3.6** (`tagSys_sat_iff`).

$$(\exists \sigma : \mathbb{N} \rightarrow \mathbb{F}, \text{tagSys}_{\mathbb{F}} \Gamma t n. \text{Sat}(\sigma) \wedge \sigma(n) = \tau) \iff \exists A, \text{infer } \Gamma t = \text{some } A \wedge \tau = \text{tag}_{\mathbb{F}}(A).$$

Read left to right this is soundness — a satisfying assignment forces the term to be well typed, at the type its root wire names. Read right to left it is completeness — every accepted term has a proof. No junk, no gaps.

## 4 Phase II: polynomial systems, flattening, gate circuits

**Definition 4.1.** `Poly  $\mathbb{F}$`  is the inductive type of multivariate polynomial expressions (`const`, `var`, `+`, `·`, negation), with `eval  $\sigma$` , a predicate `Bounded  $n$`  and a predicate `VarsIn  $S$` . A `PolySystem` is a list of polynomials, satisfied by  $\sigma$  when each evaluates to 0.

A `Gate` is `constG  $w c$`  ( $\sigma w = c$ ), `addG  $w u v$`  ( $\sigma w = \sigma u + \sigma v$ ), `mulG  $w u v$`  ( $\sigma w = \sigma u \cdot \sigma v$ ) or `zeroG  $w$`  ( $\sigma w = 0$ ). A `GateCircuit` is a list of gates, satisfied when all of them hold.

**Definition 4.2** (Tseitin flattening). `flat : Poly  $\mathbb{F}$   $\rightarrow \mathbb{N} \rightarrow \text{FlatOut } \mathbb{F}$`  compiles a polynomial starting from scratch wire  $n$ , returning an output wire, the next free wire, and the gates introduced. Variables compile to themselves and consume no scratch wire.

The scratch-wire discipline is proved, not assumed: `Poly.le_flat_next` ( $n \leq (\text{flat } p n).\text{next}$ ), `Poly.flat_out_lt` and `Poly.flat_gates_lt` (every wire the compilation touches lies below the returned frontier). This is what rules out the classic flattening bug in which two subcircuits silently share a scratch wire.

**Theorem 4.3** (`flatSys_sat_iff`). *Let  $S$  be a polynomial system with all variables below  $n$ . Then*

$$S.\text{Sat}(\sigma) \iff \exists \sigma', (\forall i < n, \sigma'(i) = \sigma(i)) \wedge (S.\text{flatSys } n).\text{Sat}(\sigma').$$

Both directions matter: right to left is soundness of the compiled artifact (a satisfying assignment for the gates restricts to one for the system), and left to right is completeness (`Poly.flat_extend` extends any solution of the system to the scratch wires).

| Name                       | Kind           | Constraint            | Correctness                        |
|----------------------------|----------------|-----------------------|------------------------------------|
| <code>isZeroSys</code>     | function       | inverse hint wire     | <code>isZeroSys_sat_iff</code>     |
| <code>muxSys</code>        | function       | $o = bx + (1 - b)y$   | <code>muxSys_selects</code>        |
| <code>hornerSys</code>     | function       | Horner trace          | <code>hornerValue_eq_sum</code>    |
| <code>rootSys</code>       | relation       | $p(w) = 0$            | <code>catalog_rootSys_roots</code> |
| <code>merklePathSys</code> | data structure | algebraic compression | <code>merklePathSys_sat_iff</code> |

Table 1: The verified gadget catalog `ZkPoP.Circ.catalog`. Its shape is pinned down by `catalog_names`, `catalog_kinds`, `catalog_names_nodup` and `catalog_length = 5`.

## 5 The verified gadget catalog

Two entries deserve comment.

*Zero testing* cannot be expressed by polynomial constraints on the input alone; `isZeroSys` therefore takes an auxiliary *hint* wire holding an inverse, and the correctness statement is a genuine “if and only if”, so a prover cannot use the hint to lie in either direction.

*Horner evaluation* is proved to compute what it claims: `hornerValue_eq_sum` states that the trace really equals  $\sum_i c_i t^i$ , and the catalog instance of `rootSys` is the concrete cubic  $X^3 - 6X^2 + 11X - 6$ , for which

$$(\text{rootSys}[-6, 11, -6, 1]0).\text{Sat}(\sigma) \iff \sigma(0) \in \{1, 2, 3\}$$

is proved over  $\mathbb{Q}$  (`catalog_rootSys_roots`). The gadget is not merely well-formed; it is solved.

## 6 Phase III: commitment and extraction

Let  $G$  be a module over the field  $\mathbb{F}$ , written additively.

**Definition 6.1.**  $\text{ped}(g, h, m, r) = m \cdot g + r \cdot h$ .

**Theorem 6.2** (Perfect hiding; `pedersen_hiding`). *If  $r \mapsto r \cdot h$  is a bijection  $\mathbb{F} \rightarrow G$ , then for all  $m, m', r$  there is a unique  $r'$  with  $\text{ped}(g, h, m, r) = \text{ped}(g, h, m', r')$ .*

So a commitment is consistent with every candidate message, under exactly one blinding factor: an observer learns nothing.

### 6.1 Hiding, unconditionally

The bijectivity hypothesis is not an assumption about the world; in the standard setting it is a theorem, and it is now proved rather than assumed.

**Theorem 6.3** (`smul_bijective_of_finrank_one`). *If  $\dim_{\mathbb{F}} G = 1$  and  $h \neq 0$ , then  $r \mapsto r \cdot h$  is a bijection  $\mathbb{F} \rightarrow G$ .*

That is exactly the deployed situation: a cyclic group of prime order  $p$  is a one-dimensional line over  $\mathbb{F} = \mathbb{Z}/p$ , and  $h \neq 0$  says the blinding generator is not the identity. Hence

**Theorem 6.4** (Unconditional hiding; `pedersen_hiding_of_finrank_one`, `pedersen_hiding_zmod`). *If  $\dim_{\mathbb{F}} G = 1$  and  $h \neq 0$ , then for all  $m, m', r$  there is a unique  $r'$  with  $\text{ped}(g, h, m, r) = \text{ped}(g, h, m', r')$ . In particular this holds in  $\mathbb{Z}/p$  for prime  $p$  and any  $h \neq 0$ , with no further hypothesis.*

The same fact has a distributional form, which is perfect hiding as it is usually stated. Over a finite field:

**Theorem 6.5** (`pedersen_fiber_card`, `pedersen_hiding_uniform`). For every message  $m$  and every group element  $c$ ,

$$|\{r \in \mathbb{F} : \text{ped}(g, h, m, r) = c\}| = 1,$$

so for all  $m, m'$  the two fibres have the same cardinality: the distribution of the commitment under a uniformly random blinding factor does not depend on the message. `pedersen_hiding_uniform_zmod` states it in  $\mathbb{Z}/p$  with  $h \neq 0$  as the only hypothesis.

The audit theorem `artifact_hides_witness_unconditional` carries this through to the published artifact: hiding of the commitment together with Shamir privacy of the shards, with no bijectivity hypothesis.

**Theorem 6.6** (Binding as a reduction; `pedersen_binding`). If  $\text{ped}(g, h, m, r) = \text{ped}(g, h, m', r')$  with  $m \neq m'$ , then

$$g = \frac{r' - r}{m - m'} \cdot h.$$

That is the statement in its exact form: a double opening *is* a discrete logarithm, exhibited. The reduction is a theorem and needs no assumption — but a reduction alone does not say that binding *holds*. For that, the assumption it reduces to has to be written down.

## 6.2 Binding as a security statement

A binding adversary is a function  $A : G \times G \rightarrow (\mathbb{F} \times \mathbb{F})^2$  taking public parameters to two claimed openings; a discrete-logarithm adversary is a function  $B : G \times G \rightarrow \mathbb{F}$ . Write

$$\text{Wins}(A, p) \iff m \neq m' \wedge \text{ped}(p_1, p_2, m, r) = \text{ped}(p_1, p_2, m', r'), \quad \text{Solves}(B, p) \iff p_1 = B(p) \cdot p_2,$$

where  $((m, r), (m', r')) = A(p)$ . The reduction of the previous theorem becomes a map on adversaries,

$$\text{red}(A)(p) = \frac{r' - r}{m - m'},$$

and `Binding.reduction_correct` says  $\text{Wins}(A, p) \Rightarrow \text{Solves}(\text{red}(A), p)$ .

**Definition 6.7** (The assumption; `Binding.DLogHard`). For a class  $\mathcal{A}$  of feasible discrete-logarithm adversaries and a set  $P$  of admissible parameters,

$$\text{DLogHard}(\mathcal{A}, P) \iff \forall B \in \mathcal{A}, \forall p \in P, \neg \text{Solves}(B, p).$$

**Theorem 6.8** (Binding; `pedersen_binding_secure`). If  $\text{red}$  maps the class  $\mathcal{B}$  of feasible binding adversaries into  $\mathcal{A}$ , and  $\text{DLogHard}(\mathcal{A}, P)$  holds, then no  $A \in \mathcal{B}$  wins at any  $p \in P$ .

The assumption is a hypothesis, not a theorem — that is the point of stating it — but it is neither vacuous nor free. It is satisfiable, and satisfiable non-trivially: `dlogHard_prod` shows that in  $\mathbb{F} \times \mathbb{F}$  with  $g = (1, 0)$  and  $h = (0, 1)$  no discrete logarithm exists at all, so the assumption holds against *every* adversary and binding is absolute there (`binding_secure_prod`); `dlogHard_univ_iff` identifies the assumption against all adversaries with the statement that  $g \notin \mathbb{F} \cdot h$ . And it is necessary: `binding_fails_when_dlog_easy` exhibits parameters ( $g = h = 1$  in  $\mathbb{F}$ ) and a concrete adversary that opens one commitment to both 0 and 1. Without some assumption on the parameters, binding is false. The audit theorem `artifact_binds_witness_secure` states the conclusion for the published artifact.

**Definition 6.9** (Schnorr). A transcript is  $(t, c, z, w)$ . The announcement is  $t = a \cdot g + b \cdot h$ ; the response to a challenge  $c$  is  $(a + cm, b + cr)$ ; verification is  $z \cdot g + w \cdot h = t + c \cdot C$ .

**Theorem 6.10** (Completeness; `Schnorr.completeness`). *An honest prover holding an opening  $(m, r)$  of  $C$  passes verification for every challenge.*

**Theorem 6.11** (Special soundness; `Schnorr.special_soundness`). *If two transcripts  $T, T'$  both verify against  $C$ , share their announcement ( $T.t = T'.t$ ) and differ in challenge ( $T.c \neq T'.c$ ), then*

$$C = \text{ped}\left(g, h, \frac{z - z'}{c - c'}, \frac{w - w'}{c - c'}\right),$$

*so the extractor `Schnorr.extract` returns an opening of  $C$ . Applied to two honest transcripts it returns the original witness (`Schnorr.extract_honest`).*

## 7 Phase IV: self-hosting and recursion

**Definition 7.1.** `checkCircuit C u` is the boolean gate interpreter, and `univCircuit C u = boolCircuit(checkCircuit C u)` where `boolCircuit(true)` has no constraints and `boolCircuit(false)` is the contradictory circuit  $\{\sigma_0 = 1, \sigma_0 = 0\}$ .

**Theorem 7.2** (`univ_sound`).  $(\text{univCircuit } C u).\text{Sat}(\sigma) \iff C.\text{Sat}(u)$ , for every outer assignment  $\sigma$ .

The universal circuit is therefore satisfiable exactly when the inner claim holds — not merely when the prover says so.

**Theorem 7.3** (Two levels; `univ_two_level`).  $(\text{univCircuit } (\text{univCircuit } C u) v).\text{Sat}(w) \iff C.\text{Sat}(u)$ ; and recursion to any depth collapses to the inner claim (`univIterate_sat`).

**Theorem 7.4** (Aggregation; `library_checked`). For a library  $L$  of circuit/assignment pairs,  $(\text{aggregate } L).\text{Sat}(\sigma) \iff \forall (C, u) \in L, C.\text{Sat}(u)$ . One proof certifies a whole library.

Self-hosting would be worthless if the universal verifier accepted everything. It does not: `demo_accepted` exhibits a concrete circuit and a valid inner assignment that is accepted for every outer assignment, and `demo_rejected` exhibits an invalid one that is rejected for every outer assignment.

## 8 Phase V: deployment, durability, audit

### 8.1 Sharding

Let  $f \in \mathbb{F}[X]$  carry the artifact and  $\text{shard}(f, x) = f(x)$ .

**Theorem 8.1** (Durability; `Shards.shards_recover`). *If  $\deg f < |s|$  for a finite carrier set  $s$ , then Lagrange interpolation on the shards held by  $s$  returns  $f$  exactly.*

**Theorem 8.2** (Privacy; `Shards.shards_hide_payload`). *Let  $|s| + 1 = N$  and let  $o \notin s$ . For every target value  $y$  there is  $g \in \mathbb{F}[X]$  with  $\deg g < N$ , agreeing with  $f$  on every shard in  $s$ , and with  $g(o) = y$ .*

Any  $N - 1$  shards are therefore consistent with *every* value of the secret: the hypothesis  $o \notin s$  is exactly the condition that the secret index is not among the surviving shares.

## 8.2 The end-to-end theorem

**Definition 8.3.**  $\text{skillCircuit } \Gamma t = (\text{tagSys}_{\mathbb{Q}} \Gamma t 0).\text{flatSys } |t|$ : the tag wires occupy  $[0, |t|)$  and scratch wires start at  $|t|$ .

**Theorem 8.4** (`end_to_end_recursive`). *If  $\text{infer } \Gamma t = \text{some } A$  then there is  $\sigma$  with*

- (i)  $(\text{tagSys}_{\mathbb{Q}} \Gamma t 0).\text{Sat}(\sigma)$  and  $\sigma(0) = \text{tag}_{\mathbb{Q}}(A)$ ;
- (ii) an extension  $w$  of  $\sigma$  with  $(\text{skillCircuit } \Gamma t).\text{Sat}(w)$ ;
- (iii) for all  $u, v$ :  $(\text{univCircuit}(\text{univCircuit}(\text{skillCircuit } \Gamma t) w) u).\text{Sat}(v)$ .

**Theorem 8.5** (`No junk`; `skillCircuit_sound`). *Conversely, if  $(\text{skillCircuit } \Gamma t).\text{Sat}(w)$  for some  $w$ , then  $\text{infer } \Gamma t = \text{some } A$  for some  $A$ .*

Together these say the artifact is a faithful certificate: it exists for every well-typed skill and for no other.

## 8.3 The audits

**Theorem 8.6** (`artifact_hides_witness`). *The published commitment can be opened to any other message under a unique blinding factor, and any  $N - 1$  shards are consistent with every value at the excluded secret index. Hiding and privacy hold simultaneously.*

**Theorem 8.7** (`artifact_binds_witness`). *A prover answering two different challenges for one announcement really knows an opening of the commitment, and a prover opening one commitment two ways has solved a discrete logarithm.*

## 8.4 A worked skill

The demo skill is `if isZero( $x + 3$ ) then 1 else  $x \cdot 2$`  in the context  $x : \text{nat}$ . It type-checks (`demoSkill_typed`, by `rfl`), evaluates at that type (`demoSkill_evaluates`, via Theorem 2.8), runs the whole pipeline (`demoSkill_end_to_end`, via Theorem 8.4) and appears in an aggregated library proof (`demo_library_checked`). The ill-typed variant, whose two branches have different types, is rejected by the checker (`demoBadSkill_untyped`, by `rfl`).

## 8.5 Axiom closure

Every headline theorem was checked with `#print axioms`. Each depends only on Lean’s standard axioms.

| Theorem                                             | Axiom closure                                                                  |
|-----------------------------------------------------|--------------------------------------------------------------------------------|
| <code>Ty.tag_injective</code>                       | <code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code> |
| <code>Ty.wires_injective_flat</code>                | <code>propext</code>                                                           |
| <code>Tm.checker_sound</code>                       | <code>propext</code> , <code>Quot.sound</code>                                 |
| <code>typeSat_iff</code>                            | <code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code> |
| <code>tagSys_sat_iff</code>                         | <code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code> |
| <code>flatSys_sat_iff</code>                        | <code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code> |
| <code>Pedersen.pedersen_hiding</code>               | <code>propext</code> , <code>Quot.sound</code>                                 |
| <code>Pedersen.pedersen_binding</code>              | <code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code> |
| <code>Pedersen.smul_bijective_of_finrank_one</code> | <code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code> |
| <code>Pedersen.pedersen_hiding_zmod</code>          | <code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code> |
| <code>Pedersen.pedersen_hiding_uniform</code>       | <code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code> |
| <code>Binding.pedersen_binding_secure</code>        | <code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code> |
| <code>Binding.binding_fails_when_dlog_easy</code>   | <code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code> |
| <code>Schnorr.special_soundness</code>              | <code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code> |
| <code>univ_two_level</code>                         | <code>propext</code> , <code>Quot.sound</code>                                 |
| <code>library_checked</code>                        | <code>propext</code> , <code>Quot.sound</code>                                 |
| <code>Shards.shards_recover</code>                  | <code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code> |
| <code>Shards.shards_hide_payload</code>             | <code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code> |
| <code>artifact_hides_witness</code>                 | <code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code> |
| <code>end_to_end_recursive</code>                   | <code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code> |

No additional axiom, no `sorry`, and no `@[implemented_by]` appears in the development.

## 9 The development as a description-logic knowledge base

### 9.1 Why

The theorems above say what the pipeline guarantees. They do not, by themselves, say what the pipeline *is* at the level a reviewer, an auditor or an ontology consumer reasons at: that an artifact is a circuit, that circuits are made of gates, that a commitment hides and binds, that a library of accepted members is accepted, that an audited theorem depends only on standard axioms. Those are terminological claims, and the natural home for terminological claims is a description logic.

The project already contains a general  $\mathcal{ALC}$  formalization (`RequestProject.ALC`): concepts  $\top, \perp, \neg C, C \sqcap D, C \sqcup D, \forall r.C, \exists r.C$ ; interpretations  $\mathcal{I} = (\Delta^{\mathcal{I}}, \mathcal{I})$ ; satisfaction of general concept inclusions and of ABox assertions; models; entailment  $\mathcal{K} \models \alpha$  and  $\mathcal{K} \models C \sqsubseteq D$ ; and consistency. Module `RequestProject.ZkPoPDL` instantiates it on this development.

### 9.2 The TBox

The signature has 41 concept names, 22 role names and 38 individual names. The TBox has 31 axioms; the significant ones are:

| Axiom                                                                                                                         | Renders                                                   |
|-------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------|
| <code>WellTyped <math>\sqsubseteq</math> Skill <math>\sqcap</math> <math>\exists</math>hasType.Type</code>                    | <code>infer <math>\Gamma t = \text{some } A</math></code> |
| <code>WellTyped <math>\sqsubseteq</math> <math>\exists</math>evaluatesTo.Value</code>                                         | <code>Tm.checker_sound</code>                             |
| <code>IllTyped <math>\sqsubseteq</math> <math>\neg</math>WellTyped</code>                                                     | <code>demoBadSkill_untyped</code>                         |
| <code>WellTyped <math>\sqsubseteq</math> <math>\exists</math>hasArith.(Arithmetization <math>\sqcap</math> PolySystem)</code> | <code>tagSys_sat_iff</code>                               |

| Axiom                                                                                                                                              | Renders                                |
|----------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------|
| $\text{PolySystem} \sqsubseteq \exists \text{compilesTo.Circuit}$                                                                                  | <code>flatSys_sat_iff</code>           |
| $\text{Gadget} \sqsubseteq \text{Circuit} \sqcap \text{Sound} \sqcap \text{Complete}$                                                              | the verified catalog                   |
| $\text{Artifact} \sqsubseteq \text{Circuit} \sqcap \text{Sound} \sqcap \exists \text{commitsTo.Commitment} \sqcap \exists \text{shardsInto.Shard}$ | <code>skillCircuit</code> , deployment |
| $\text{Commitment} \sqsubseteq \text{Hiding} \sqcap \text{Binding}$                                                                                | Pedersen                               |
| $\text{Hiding} \sqsubseteq \neg \exists \text{reveals.Secret}$                                                                                     | <code>pedersen_hiding</code>           |
| $\text{Hiding} \sqsubseteq \text{Unconditional}$                                                                                                   | <code>pedersen_hiding_zmod</code>      |
| $\text{Binding} \sqsubseteq \exists \text{reducesTo.DLogProblem}$                                                                                  | <code>Binding.reduction_correct</code> |
| $\text{DLogProblem} \sqsubseteq \exists \text{assumes.HardnessAssumption}$                                                                         | <code>Binding.DLogHard</code>          |
| $\text{HardnessAssumption} \sqsubseteq \neg \text{Theorem} \sqcap \neg \text{Unconditional}$                                                       | hardness is assumed                    |
| $\text{Theorem} \sqcap \exists \text{assumes.HardnessAssumption} \sqsubseteq \neg \text{Unconditional}$                                            | what rests on an assumption            |
| $\text{Shard} \sqsubseteq \neg \exists \text{reveals.Secret}$                                                                                      | <code>shards_hide_payload</code>       |
| $\text{Artifact} \sqcap \exists \text{satisfiedBy.Witness} \sqsubseteq \exists \text{certifies.WellTyped}$                                         | <code>skillCircuit_sound</code>        |
| $\text{Verifier} \sqcap \text{Universal} \sqsubseteq \text{Circuit} \sqcap \text{Recursive}$                                                       | self-hosting                           |
| $\text{Recursive} \sqsubseteq \exists \text{verifies.Circuit}$                                                                                     | <code>univ_two_level</code>            |
| $\text{Transcript} \sqcap \exists \text{forks.Transcript} \sqsubseteq \exists \text{extractedBy.Extractor}$                                        | <code>special_soundness</code>         |
| $\text{Extractor} \sqsubseteq \exists \text{yields.Witness}$                                                                                       | <code>Schnorr.extract</code>           |
| $\text{Library} \sqcap \forall \text{aggregates.Accepted} \sqsubseteq \text{Accepted}$                                                             | <code>library_checked</code>           |
| $\text{Audited} \sqsubseteq \text{Theorem} \sqcap \forall \text{dependsOn.StandardAxiom}$                                                          | the axiom-closure audit                |
| $\text{Circuit} \sqsubseteq \text{MathObject}$                                                                                                     | —                                      |
| $\text{MathObject} \sqsubseteq \exists \text{canServe.Sword} \sqcap \exists \text{canServe.GreenSilence}$                                          | the poem cycle’s dual-use axiom        |

The ABox has 95 assertions naming the concrete objects of the development: the demo skill and its ill-typed twin, the tag system, the compiled artifact and its witness, the five catalog gadgets with their kinds, the commitment, a shard, the Schnorr protocol with two forking transcripts and its extractor, two levels of universal verifier, the library, three audited theorems and the three standard axioms they depend on, the discrete-logarithm problem and the hardness assumption the binding theorem rests on, plus distinctness assertions. Among the entailments proved from this base are  $\text{Commitment} \sqsubseteq \text{Unconditional} \sqcap \exists \text{reducesTo.DLogProblem}$  (hiding is unconditional, binding is a reduction),  $\text{Binding} \sqsubseteq \exists \text{reducesTo}.\exists \text{assumes.HardnessAssumption}$ , and  $\text{bindsThm} : \neg \text{Unconditional}$  — the DL layer records, as the Lean layer does, which of the two guarantees is free and which is bought.

### 9.3 Consistency

**Theorem 9.1** ( $\text{ZkPoPDL.zkKB\_consistent}$ ). *The knowledge base is consistent: an explicit finite interpretation — domain the set of individual names, each interpreted by itself — satisfies all 31 TBox axioms and all 95 ABox assertions. Satisfaction is decided by a decision procedure for concept membership in that model, so the verification is by computation.*

Consistency matters here for a specific reason: an inconsistent knowledge base entails everything, so without this theorem the entailments below would be worthless.

## 9.4 Entailments

Each of the following is proved to hold in *every* model of the knowledge base.

- `wellTyped_evaluates`:  $\text{WellTyped} \sqsubseteq \exists \text{evaluatesTo.Value}$ .
- `wellTyped_compiles`:  $\text{WellTyped} \sqsubseteq \exists \text{hasArith.}(\text{PolySystem} \sqcap \exists \text{compilesTo.Circuit})$  — Phases I and II chained.
- `artifact_certifies_wellTyped`: the artifact certifies a well-typed skill (no junk).
- `artifact_accepted`: the artifact is accepted, hence not rejected.
- `artifact_hides`:  $\text{Artifact} \sqsubseteq \exists \text{commitsTo.} \neg \exists \text{reveals.Secret}$ .
- `artifact_shards_hide`: the same for shards.
- `artifact_two_level`: the artifact is verified by something that is a circuit, is recursive, and verifies a circuit.
- `forking_transcript_extracts`: a forking transcript is extracted by an extractor that yields the witness.
- `library_accepted`: a library all of whose members are accepted is accepted and not rejected.
- `endToEnd_standard_axioms`: the end-to-end theorem depends only on standard axioms.
- `gadgets_sound_complete`: every catalog gadget is a sound, complete circuit with a kind.
- `artifact_dual_use`:  $\text{Artifact} \sqsubseteq \exists \text{canServe.Sword} \sqcap \exists \text{canServe.GreenSilence}$ .

## 9.5 Non-entailments

Four statements are proved *not* to follow, each witnessed by the finite model:

- `not_entails_artifact_serves_sword` and `not_entails_artifact_serves_greenSilence`: the knowledge base entails that the artifact *can* serve either master, and entails neither that it does. In the vocabulary of the poem cycle: possibility is not deed.
- `not_entails_badSkill_accepted`: the ill-typed skill is not accepted — the verifier is not blind.
- `not_entails_artifact_reveals`: nothing forces the artifact to reveal a secret; hiding is a theorem of the base, not an assumption.

## 9.6 Reality check

The individuals are not free-floating. `demoSkill_reality` proves that the individual *demoSkill* denotes a term the real checker accepts at `nat`, that *badSkill* denotes one the real checker rejects, and that the former really evaluates to a value of that type. `skillCirc_reality` proves that the artifact individual denotes the real compiled circuit, satisfied by a real witness, accepted at two levels of the real universal verifier. `catalog_reality` pins the five gadget individuals to the five entries of the real catalog. The knowledge base is thus tied to the code it describes, not merely consistent on its own terms.

An `#print axioms` block at the end of `ZkPoPDL.lean` records that every result of this layer, too, depends only on `propxt`, `Classical.choice` and `Quot.sound`.

## 10 What is *not* proved

Formalization is worth little without an accurate statement of its scope.

1. **This is not a proof of a deployed zk-SNARK.** The development proves the arithmetization layer exactly: checker  $\leftrightarrow$  polynomial system  $\leftrightarrow$  gate circuit. It does not construct or verify a polynomial commitment scheme, a trusted setup, a Fiat–Shamir transformation, or a concrete proof system’s knowledge soundness in the random-oracle or algebraic-group model.
2. **Binding rests on an assumption, and the assumption is not proved.** Hiding is no longer conditional: the bijectivity hypothesis is discharged for one-dimensional spaces, hence for prime-order groups (§6.1), and the distributional form is proved over any finite field. Binding is now a security statement rather than only a reduction (§6.2), but it is stated relative to **DLogHard**, a hypothesis about a class of adversaries and a set of parameters. Hardness of the discrete logarithm in a cryptographic group is not proved — it is not known — and the adversary model is a plain function, with no complexity measure, so “feasible” is an abstract class constrained only by the closure condition of the theorem. What the development contributes here is that the assumption is written down and used, that the reduction is checked against it, and that it is shown both satisfiable and necessary.
3. **The hash is algebraic.** The Merkle gadget uses an explicit algebraic compression function, since an arithmetic circuit can only speak about algebraic hashes. Collision resistance is not proved and is not claimed.
4. **Zero-knowledge is not formalized as a simulation.** What is proved is the concrete privacy content the SOP asks for: unique-opening hiding of the commitment, and Shamir privacy of the shards. A simulator-based definition of zero knowledge is not stated.
5. **The universal verifier is a circuit, not a machine.** Its satisfiability is proved equivalent to the inner claim; that equivalence is the whole of the recursion theorem. No claim is made about proof size, prover time, or the cost of the recursion.
6. **The description-logic layer describes, and does not certify.** Section 9 is a faithful *ACC* rendering with a consistency proof and a reality check tying individuals to real objects; it adds a second, coarser view of the same development, not a second guarantee about it.

## 11 Conclusion

The pipeline is proved end to end, in one system, with no gaps: what the checker accepts, the arithmetization satisfies; what the arithmetization satisfies, the flattened circuit satisfies; what the flattened circuit satisfies, the universal verifier accepts, at two levels and at any depth; what the artifact publishes hides the witness and binds the prover; and what a satisfied artifact certifies is a well-typed skill and nothing else. The axiom closure of every one of those statements is Lean’s standard three.

The last theorem of the development is not about circuits. It says that the artifact we built can serve the sword and can serve the brief green silence, and that nothing in the mathematics decides which. The rooster of the poem cycle learns the same thing the hard way:

*They say we fight to find the truth,  
But whose truth justifies such uncouth  
Violence, whether of claw or code?  
Both rooster and Coq bear this load.*

A verifier can settle the first question completely and the second not at all. Being exact about which is which is, in the end, the only honest contribution a proof can make.

## A Module map

| Module                      | Contents                                                                                                   |
|-----------------------------|------------------------------------------------------------------------------------------------------------|
| ZkPoP/Ty.lean               | types, tag, wires, injectivity, index-normal form                                                          |
| ZkPoP/Term.lean             | terms, values, checker, evaluator, <code>checker_sound</code>                                              |
| ZkPoP/TypeSat.lean          | relational arithmetization, <code>typeSat_iff</code> , gate shapes                                         |
| ZkPoP/Circuit.lean          | <code>Poly</code> , <code>PolySystem</code> , <code>Gate</code> , flattening, <code>flatSys_sat_iff</code> |
| ZkPoP/Catalog.lean          | the five verified gadgets and the catalog metadata                                                         |
| ZkPoP/Arith.lean            | <code>tagEqs/tagSys</code> , canonical witness, <code>tagSys_sat_iff</code>                                |
| ZkPoP/Pedersen.lean         | commitment, hiding, binding, homomorphism                                                                  |
| ZkPoP/PedersenSecurity.lean | unconditional hiding, the dlog assumption, binding as security                                             |
| ZkPoP/Schnorr.lean          | $\Sigma$ -protocol, completeness, extractor, special soundness                                             |
| ZkPoP/SelfHost.lean         | universal circuit, recursion, aggregation, demos                                                           |
| ZkPoP/Shards.lean           | Reed–Solomon durability, Shamir privacy                                                                    |
| ZkPoP/Audit.lean            | <code>end_to_end_recursive</code> , privacy/binding audits, axiom closure                                  |
| ZkPoPDL.lean                | the development as an $\mathcal{ALC}$ knowledge base                                                       |
| ALC.lean                    | the description logic $\mathcal{ALC}$ over an arbitrary signature                                          |